

# Project #2: Evaluations of Vector Search Methods with FAISS

M.Sc. in Data and Web Science – Advanced Topics in Databases

## Introduction

In this project, you will design and run an experimental study of vector search methods using [FAISS](#). You will compare three methods from different families, specifically an inverted-file method with product quantization, a graph-based method with scalar quantization, and a hashing method. Concretely, you will use IVFPQ, HNSWSQ, and LSH. You will evaluate these methods on three standard datasets, SIFT, GIST, and GloVe, and study both accuracy and speed under different parameter settings. You will also analyze query hardness using Relative Contrast (RC), create harder queries through simple perturbation strategies, and observe how the behavior of the three methods changes.

## What you should measure

You should measure recall and queries per second (QPS) for every method, dataset, and parameter configuration. Recall@k measures accuracy by comparing the approximate k nearest neighbors returned by an index with the exact k nearest neighbors returned by brute-force search. If  $A_k(q)$  is the approximate result set for query  $q$  and  $G_k(q)$  is the exact ground-truth set, then:

$$\text{Recall@k}(q) = |A_k(q) \cap G_k(q)| / k$$
$$\text{Average Recall@k} = (1 / |Q|) * \sum_q \text{Recall@k}(q)$$

QPS measures throughput, that is, how many queries the system can answer per second. If you issue  $N$  queries and the total search time is  $T$  seconds, then:

$$\text{QPS} = N / T$$

A good method is not simply the one with the highest recall, the highest QPS, or lowest memory usage alone. Your analysis should emphasize the trade-off between them. For example, one method may achieve higher recall but require more time per query, while another may be much faster but need more memory (often such trade-offs are represented with accuracy-runtime curve plots)

## Introduction Relative Contrast and query hardness

To analyze query hardness, you will compute [Relative Contrast](#) (RC). For a query vector  $q$ , let  $r_k$  denote the distance from  $q$  to its  $k$ -th nearest neighbor in the database  $S$ , and let  $d_{\text{mean}}$  denote the average distance from  $q$  to all points in  $S$ . The exact formula is:

$$d_{\text{mean}} = (1 / |S|) * \sum_{x \in S} d(q, x)$$

$$RC_k(q) = d_{\text{mean}} / r_k$$

A small RC value means that the k-th nearest neighbor is not much closer than an average point in the dataset, so the query is considered hard. A larger RC value means the nearest neighbors stand out more clearly, so the query is easier. In your experiments, you should compute RC for each query, divide the queries into easy and hard groups, and compare how recall and QPS change across those groups.

## Datasets and links

You will use SIFT, GIST, and GloVe. The ANN-Benchmarks project lists all three benchmark datasets, and its dataset-generation code also shows the canonical source files used for downloading them.

| Dataset | Benchmark / Description Link                                                                                                                  | Canonical Download Source                                                                                           |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| SIFT    | <a href="https://ann-benchmarks.com/sift-128-euclidean_10_euclidean.html">https://ann-benchmarks.com/sift-128-euclidean_10_euclidean.html</a> | <a href="ftp://ftp.irisa.fr/local/texmex/corpus/sift.tar.gz">ftp://ftp.irisa.fr/local/texmex/corpus/sift.tar.gz</a> |
| GIST    | <a href="https://ann-benchmarks.com/gist-960-euclidean_10_euclidean.html">https://ann-benchmarks.com/gist-960-euclidean_10_euclidean.html</a> | <a href="ftp://ftp.irisa.fr/local/texmex/corpus/gist.tar.gz">ftp://ftp.irisa.fr/local/texmex/corpus/gist.tar.gz</a> |
| GloVe   | <a href="https://ann-benchmarks.com/glove-100-angular_10-angular.html">https://ann-benchmarks.com/glove-100-angular_10-angular.html</a>       | <a href="http://nlp.stanford.edu/data/glove.twitter.27B.zip">http://nlp.stanford.edu/data/glove.twitter.27B.zip</a> |

SIFT and GIST are Euclidean datasets. GloVe in ANN-Benchmarks is commonly treated as an angular dataset. For angular similarity in FAISS, you should L2-normalize the vectors first and then use inner product search.

## Step-by-step FAISS workflow

Your implementation should follow a clear pipeline. First, load the base vectors and query vectors into float32 NumPy arrays. Second, build an exact Flat index to produce ground truth (i.e., for each query, which vectors are the true nearest neighbors). Third, build an approximate index with the FAISS factory string, train it when required, add the base vectors, set the search parameters, and time the search. Finally, compute recall and QPS and repeat this process for multiple parameter settings.

```
import time
import numpy as np
import faiss

def l2_normalize(x):
    x = x.astype('float32', copy=False)
    faiss.normalize_L2(x)
    return x

def build_ground_truth(xb, xq, k, metric):
    gt = faiss.IndexFlatIP(xb.shape[1]) if metric == faiss.METRIC_INNER_PRODUCT else
```

```

faiss.IndexFlatL2(xb.shape[1])
gt.add(xb)
D_gt, I_gt = gt.search(xq, k)
return D_gt, I_gt

def recall_at_k(I_approx, I_gt, k):
    hits = 0
    for i in range(I_gt.shape[0]):
        hits += len(set(I_approx[i, :k]).intersection(set(I_gt[i, :k])))
    return hits / (I_gt.shape[0] * k)

def benchmark_index(index, xq, k):
    t0 = time.perf_counter()
    D, I = index.search(xq, k)
    t1 = time.perf_counter()
    total_time = t1 - t0
    qps = xq.shape[0] / total_time
    return D, I, total_time, qps

```

The following example shows how to build an index with FAISS factory, train it if needed, add vectors, and search it. For SIFT and GIST, you can use L2 distance directly. For GloVe, normalize both base and query vectors and switch to inner product.

```

# Example setup
k = 10
d = xb.shape[1]
metric = faiss.METRIC_L2    # use faiss.METRIC_INNER_PRODUCT for normalized GloVe

# Ground truth
D_gt, I_gt = build_ground_truth(xb, xq, k, metric)

# Factory example
factory = "IVF4096,PQ32x8"
index = faiss.index_factory(d, factory, metric)

if not index.is_trained:
    index.train(xb)

index.add(xb)

# Set search parameters when applicable
ps = faiss.ParameterSpace()
ps.set_index_parameter(index, "nprobe", 16)

D, I, total_time, qps = benchmark_index(index, xq, k)
recall = recall_at_k(I, I_gt, k)

print(factory, "Recall@10 =", recall, "QPS =", qps)

```

## Required methods and parameter sweeps

In addition to comparing three methods from different categories, you should tune the main structural and search parameters of each method. Your report should explain what you changed, what trade-off each parameter controls, and how the behavior changes across datasets. Keep the sweeps broad enough to reveal recall-QPS trade-offs, but small enough to remain feasible.

```

# IVF-PQ: inverted file + product quantization
factory_ivfpq = "IVF4096,PQ32x8"

```

```
# HNSWSQ: HNSW graph + scalar quantization
factory_hnswsq = "HNSW32_SQ8"

# LSH: locality-sensitive hashing
factory_lsh = "LSH"
# or use faiss.IndexLSH(d, nbits) directly
```

## IVFPQ

For IVFPQ, you should tune both the search-time parameter and the compression structure. At search time, sweep `nprobe`, which controls how many inverted lists are visited for each query. A larger `nprobe` usually improves recall but increases query time. You should also tune the number of PQ subspaces, because this changes the quantization granularity and the memory-accuracy trade-off. For a  $d$ -dimensional vector, typical choices are  $m = 8, 16$ , and  $32$ , or another divisor of  $d$ .

```
factory_ivfpq_8 = "IVF4096,PQ8x8"
factory_ivfpq_16 = "IVF4096,PQ16x8"
factory_ivfpq_32 = "IVF4096,PQ32x8"
nprobe_values = [1, 4, 8, 16, 32, 64]
```

In your discussion, separate the two effects: `nprobe` changes how much of the database is examined at query time, while the number of subspaces changes compression quality and approximation error.

## HNSWSQ

For HNSWSQ, you should tune both the graph search effort and the scalar quantization bit-width. At search time, sweep `efSearch`, which controls how many candidates are explored during graph traversal. You may also set `efConstruction` during index building. In addition, compare different scalar quantization precisions, because HNSWSQb with different  $b$  changes the compression level of stored vectors.

```
factory_hnswsq_4 = "HNSW32_SQ4"
factory_hnswsq_6 = "HNSW32_SQ6"
factory_hnswsq_8 = "HNSW32_SQ8"
efsearch_values = [16, 32, 64, 128, 256]
```

Smaller quantization bit-width reduces memory usage but increases approximation error, while larger bit-width preserves more information at a higher storage cost. A useful question is whether stronger graph search can partly compensate for more aggressive quantization.

## LSH

For LSH, you should tune the number of hash bits, or equivalently the number of projections used to form the binary code. You may use the factory string LSH for a default setup, or instantiate `faiss.IndexLSH(d, nbits)` directly so that you can control `nbits`. A larger number of hash bits usually makes the code more selective, which may improve discrimination, but it can also reduce collision probability and affect recall.

```
lsh_128 = faiss.IndexLSH(d, 128)
lsh_256 = faiss.IndexLSH(d, 256)
lsh_512 = faiss.IndexLSH(d, 512)
```

When you analyze LSH, explain how changing the number of bits affects the balance between selectivity and robustness, and compare this behavior with IVFPQ and HNSWSQ under the same recall and QPS measurements.

## Introduction How to compute RC in code

```
def compute_rc(xb, xq, k, metric):
    # exact distances only, so use a Flat index
    gt = faiss.IndexFlatIP(xb.shape[1]) if metric == faiss.METRIC_INNER_PRODUCT else
    faiss.IndexFlatL2(xb.shape[1])
    gt.add(xb)
    Dk, Ik = gt.search(xq, k)          # Dk[:, k-1] is r_k (or similarity if IP)

    # For Euclidean RC, use distances directly.
    # For inner-product datasets such as normalized GloVe, convert to distances carefully
    # or restrict RC analysis to Euclidean datasets unless you define an equivalent distance.
    rk = Dk[:, k - 1]

    dmean = []
    for q in xq:
        if metric == faiss.METRIC_L2:
            dist = ((xb - q) ** 2).sum(axis=1)
            dmean.append(dist.mean())
        else:
            dist = 1.0 - xb @ q        # after L2 normalization, cosine distance proxy
            dmean.append(dist.mean())

    dmean = np.array(dmean)
    rc = dmean / rk
    return rc
```

After computing RC, you may sort queries by RC and divide them into quantiles, such as the bottom 25 percent for hard queries and the top 25 percent for easy queries. You should then report recall and QPS separately for these groups.

## How to create harder queries

You should also design a simple strategy to make queries harder and then observe how each method responds. One straightforward approach is to add small Gaussian noise to the query vectors. Another approach is to interpolate a query toward a non-neighbor point so that the distance gap between the nearest neighbors and the rest becomes smaller. You may also compare clean queries with perturbed queries under several noise levels.

```
def add_noise(xq, sigma):
    noisy = xq + sigma * np.random.randn(*xq.shape).astype('float32')
    return noisy.astype('float32')

for sigma in [0.00, 0.01, 0.05, 0.10]:
    xq_noisy = add_noise(xq, sigma)
    if metric == faiss.METRIC_INNER_PRODUCT:
        xq_noisy = l2_normalize(xq_noisy)
    D, I, t, qps = benchmark_index(index, xq_noisy, k)
    print("sigma =", sigma, "recall =", recall_at_k(I, I_gt, k), "qps =", qps)
```

## Deliverables (**last week of the semester, May 28<sup>th</sup>**)

- Your code and, if applicable, the Jupyter Notebook, including exported PDF and HTML formats
- Your datasets along with modified queries to reflect different query hardness
- A README with instructions for replicating your results/plots
- A poster A0 style (essentially ~10 slides integrated in a poster format), explaining the problem, methods, setup, datasets, and modifications. You should discuss which method gives the best recall-QPS trade-off on each dataset, whether the best method changes across SIFT, GIST, and GloVe, and how robust each method is when queries become harder. You should also comment on whether RC is a useful predictor of search difficulty, and whether simple adaptation strategies such as increasing nprobe or efSearch for hard queries improve results
- The poster will be presented in the last class of the semester in a “poster + pizza event” that we will organize.